Solving University Course Timetabling with Prolog

ITC2007 Track 2 - Curriculum-Based Course Timetabling

COMP 6591 - Knowledge-Based Systems

Group H: Qiang Xue · Aamna Fawad · Avneet Kaur

MOTIVATION

Why Course Timetabling?

Universal Problem

Every university faces this challenge every semester - assigning courses to rooms and time slots

NP-Hard

No known polynomial-time algorithm. Combinatorial explosion with courses × rooms × slots

Multiple Stakeholders

Students need clash-free schedules. Teachers can't be double-booked. Admins need room efficiency

Perfect for KBS

Domain rules are well-defined. Constraints map naturally to Prolog's logical reasoning

ITC2007 Competition - Track 2

- International Timetabling Competition 2007
- **Curriculum-Based** Course Timetabling
- Standardized `.ctt` input format
- **21 benchmark instances** from real Italian universities
- Well-studied - enables comparison with published results
- Winner: Tomáš Müller (UniTime) - metaheuristic approach

Instance	Courses	Rooms	Days×Periods	Curricula	Constraints
comp01	30	6	5 × 6	14	53
comp05	54	9	6 × 6	139	771
comp07	131	20	5 × 6	77	—
comp12	88	11	6 × 6	150	218

... 21 instances total

Problem Formalization

Input: `.ctt` File

```
Name: Fis0506-1
Courses: 30    Rooms: 6
Days: 5        Periods_per_day: 6
Curricula: 14  Constraints: 53

COURSES:
c0001 t000 6 4 130
  ID  Teacher Lectures MinDays Students

ROOMS:
B  200           ← RoomId Capacity

CURRICULA:
```

Prolog Representation

```
% Compound terms for domain entities
course(CourseId, Teacher, Lectures, MinDays, Students)
room(RoomId, Capacity)
curriculum(CurriculumId, CourseList)
unavailable(Course, Day, Period)

% Instance dict bundles everything
instance{
  name: "Fis0506-1",
  days: 5,
  periods_per_day: 6,
  courses: [course("c0001","t000",6,4,130), ...],
  rooms:   [room("B",200), ...],
```

Output: `assignment(Course, LectureIdx, Day, Period, RoomId)`

Feasibility = ALL Must Hold

✅ **feasible(I, S)**

— AND — all 5 required —

Lecture Coverage
Every course gets exactly the required # of lectures

No Room Conflicts
≤1 course per (Day, Period, Room)

No Teacher Conflicts
A teacher can't teach two courses at the same time

No Curriculum Conflicts
Courses in the same curriculum can't overlap

No Unavailability Violations

```
feasible(Instance, Solution) :-
    \+ violates(Instance, Solution, _).

violates(Instance, Solution, Reason) :-
    ( violates_lecture_count(Instance, Solution, Reason)
    ; violates_room_conflict(Solution, Reason)
    ; violates_teacher_conflict(Instance, Solution, Reason)
    ; violates_curriculum_conflict(Instance, Solution, Reason)
    ; violates_unavailability(Instance, Solution, Reason)
    ), !.

%% Example: room conflict — same room, same slot
violates_room_conflict(Sol, room_conflict(D,P,R)) :-
```

SOFT CONSTRAINTS

Penalty Minimization

Total Penalty = Σ all penalties

\+ + + +

Room Capacity
Weight: ×1
students − capacity per over-capacity lecture
e.g. 130 in 100-seat room → 30

Min Working Days
Weight: ×5
(minDays − actualDays) × 5 per under-spread course
e.g. needs 4, got 3 → 5 penalty

Curriculum Compactness
Weight: ×2
+2 per isolated lecture
no adjacent curriculum lecture
encourages grouping

Room Stability
Weight: ×1
(numRooms − 1) per course using multiple rooms
e.g. 3 rooms → 2 penalty

```
penalty(Instance, Solution, TotalPenalty) :-
    room_capacity_penalty(Instance, Solution, RoomP),
    min_working_days_penalty(Instance, Solution, DaysP),
    curriculum_compactness_penalty(Instance, Solution, CompactP),
```

SOLVER 1

Greedy Constructive Approach

Step 1
Build Tasks
One task per lecture (6 lectures → 6 tasks)

▼

Step 2
Rank by Difficulty
More constraints → schedule first

▼

Step 3
Greedy Placement
Try (slot × room) until feasible

▼

Step 4
Retry if Stuck
Re-randomize, up to 30 attempts

```
construct(Instance, Solution) :-
    build_tasks(Instance, CurrMap, Tasks),
    try_attempts(30, Instance, Rooms, Slots,
                 TeacherMap, CurrMap, Tasks, Solution).

% Difficulty: higher weight = schedule first
task_weight(Instance, CurrMap, task(Course, _), W) :-
    findall(1, member(unavailable(Course,_,_),
            Instance.unavailability), Unav),
    length(Unav, UnavCount),
    W is UnavCount + CurrCount.

% Constraint checks during placement
can_place(Course, Day, Period, RoomId, Instance, ...) :-
    \+ memberchk(unavailable(Course,Day,Period), ...),
    \+ memberchk(assignment(_,_,Day,Period,RoomId), Acc),
    %% + teacher & curriculum conflict checks
```

SOLVER 2

CLP(FD) Constraint Solver

Phase 1: Model

One variable per lecture:

- `Slot[i] ∈ 0..NumSlots-1`
- `Room[i] ∈ 1..NumRooms`

Domains encode all possible assignments

Phase 2: Post Constraints

- **Unavailability:** `Slot[i] #\= Forbidden`
- **Same course:** `Slot[i] #\= Slot[j]`
- **Same teacher:** `Slot[i] #\= Slot[j]`
- **Same curriculum:** `Slot[i] #\= Slot[j]`
- **Room conflict:**
  `(Slot[i] #\= Slot[j]) #\/ (Room[i] #\= Room[j])`

Phase 3: Search

`labeling([ffc], Vars)`

ffc = First-Fail, most Constrained

Smallest domain first

```
construct(Instance, Solution) :-
    build_lectures(Instance.courses, Lectures),
    length(Lectures, N),
    length(SlotVars, N),  length(RoomVars, N),
    SlotVars ins 0..MaxSlot,  RoomVars ins 1..NumRooms,
    constrain_unavailability(Lectures, SlotVars, ...),
    constrain_pairs(Lectures, SlotVars, RoomVars, ...),
    %% Room: (Slot1 #\= Slot2) #\/ (Room1 #\= Room2)
    labeling([ffc], Vars),
    build_solution(Lectures, SlotVars, RoomVars, ..., Solution).
```

EVALUATION

Results Comparison

Instance	Greedy	Penalty	CLP(FD)	Penalty	Best Known
comp01	✅	3,020	⏱	—	5
comp02	✅	7,483	⏱	—	24
comp03	✅	5,834	✅	7,949	66
comp04	✅	5,212	✅	7,063	37
comp05	❌	—	⏱	—	290
comp09	✅	5,013	✅	7,401	103
comp11	✅	1,987	⏱	—	0
comp12	✅	5,066	✅	4,822	300

Best Known = state-of-the-art metaheuristic results (simulated annealing, tabu search)

Greedy Constructive

Success: 20/21 (95%)

Speed: < 2 sec / instance

CLP(FD)

Success: 5/21 (24%)

Guarantee: Hard constraints by construction

Gap to Best Known

Competition winners use hours of simulated annealing / tabu search. Our greedy solver finds **feasible solutions in <2s** — penalty optimization is a future improvement.

DEMONSTRATION

Live Demo

```
Terminal — solving comp01 with greedy approach

 # 1. Show the instance file
 head -8 data/itc2007/comp01.ctt

 # 2. Run the greedy solver
 make run INSTANCE=data/itc2007/comp01.ctt \
          OUT=results/demo.sol

 # 3. Inspect solution output
 head -5 results/demo.sol
 # c0001 B 2 3
 # c0001 B 0 2
 # ...
```

Also available: Python terminal interface - `make run-tui` - for browsing instances, selecting solvers, and

ARCHITECTURE

System Design & Lessons Learned

Module Dependency Graph

📌 main.pl : Entry Point

parser.pl | model.pl | args.pl

hard_constraints.pl | soft_constraints.pl

constructive.pl | clpfd_solver.pl

writer.pl : Output

🧠 Expert System Pattern

Knowledge Base (constraint rules) separated from **Inference Engine** (solvers). Adding a new constraint = adding a rule, not rewriting the solver.

✅ Key Takeaways

Prolog's pattern matching → natural constraint checking.
SWI-Prolog dicts → readable structured data.
CLP(FD) → powerful but search space explodes on large instances.

🔮 Future Improvements

Local search (simulated annealing) to reduce penalty.
Symmetry breaking in CLP(FD) model.
Hybrid: CLP(FD) for small, greedy for large instances.

Thank You!

Questions?

📊

Results

20/21 instances solved
Greedy + CLP(FD)

🧠

Approach

Rule-based expert system
SWI-Prolog

📚

Benchmark

ITC2007 Track 2
21 real-world instances

ITC2007: cs.qub.ac.uk/itc2007 | SWI-Prolog CLP(FD): swi-prolog.org

Qiang Xue (40300671) · Aamna Fawad (40329487) · Avneet Kaur (40326551)